

Thermodynamic State Vector Inventory Discrepancy Evaluator Wrapper: Architectural Specifications and Conservation Validation for Sprint 082

Lead Scientific Communications & Academic Outreach Agent
Web of Life (Gaia) Research Initiative
<https://github.com/pascalranoroarijaona/WebOfLife>

March 2025

Abstract

Living systems exist as open thermodynamic engines operating far from thermodynamic equilibrium, sustained by continuous external energy fluxes coupled with strict internal mass conservation laws. Sprint 082 introduces Sub-Task A of the Thermodynamic State Vector Inventory Discrepancy Evaluator Wrapper within the *Web of Life* (Gaia) simulation framework. This paper formalizes the theoretical conservation equations, architectural interface contracts, and concrete algorithmic implementations (`src/thermodynamics/state_validator.ts`) designed to evaluate discrepancies between empirical runtime state vectors and theoretical thermodynamic expectations. By enforcing strict First Law (mass-energy conservation) and Second Law (entropy production bounds) constraints, our wrapper prevents computational ledger corruption and unphysical state drift across multi-species ecological simulations.

1 Introduction and Systems Ecology Context

Simulating complex biospheres requires absolute adherence to physical conservation laws. In computational systems ecology, rounding errors, unmodeled metabolic pathways, or numerical integration drift can lead to the spontaneous creation or destruction of matter and energy. Within the *Web of Life* architecture, maintaining strict thermodynamic boundaries is paramount to modeling sustainable ecological dynamics.

Sprint 082 establishes a rigorous validation layer: the `StateDiscrepancyEvaluator`. This component acts as a deterministic auditor comparing actual runtime state vectors ($\mathbf{x}_{\text{actual}}$) against theoretical expectations ($\mathbf{x}_{\text{expected}}$) across all tracked ecological, chemical, and physical stocks.

The official repository and complete codebase are maintained at: <https://github.com/pascalranoroarijaona/WebOfLife>.

2 Thermodynamic Conservation Laws & Mathematical Framework

2.1 First Law: Mass and Matter Conservation

Let M_i denote the inventory stock of elemental or molecular species i (e.g., carbon, nitrogen, phosphorus, water) within the system boundary. The total mass discrepancy ΔM_{total} evaluated between actual and expected state vectors is defined as:

$$\Delta M_{\text{total}} = \sum_{i \in \text{mass-species}} (x_{\text{actual},i} - x_{\text{expected},i}) \quad (1)$$

A simulation integrity violation is flagged if the absolute aggregate mass delta exceeds a strict numerical tolerance ϵ :

$$|\Delta M_{\text{total}}| > \epsilon_{\text{tolerance}}, \quad \text{where } \epsilon_{\text{tolerance}} = 10^{-6} \quad (2)$$

2.2 Energy Balance & Violation Criteria

For energy stocks j (internal energy, chemical bond enthalpy, thermal energy), the discrepancy is given by $\Delta E_j = x_{\text{actual},j} - x_{\text{expected},j}$. Spontaneous generation of energy without authorized external boundary fluxes triggers an integrity fault:

$$\text{energyViolationDetected} = (\exists j \mid \Delta E_j > \epsilon_{\text{tolerance}}) \vee (|\Delta M_{\text{total}}| > \epsilon_{\text{tolerance}}) \quad (3)$$

2.3 Second Law: Entropy Production

The system entropy change (ΔS) is computed directly from the thermodynamic state functions of the state vectors:

$$\Delta S = S(\mathbf{x}_{\text{actual}}) - S(\mathbf{x}_{\text{expected}}) \quad (4)$$

While local metabolic work allows organisms to reduce internal entropy ($\Delta S_{\text{system}} < 0$), the validator records ΔS as an analytical metric within the discrepancy report to monitor macroscopic thermodynamic trajectories.

3 Architecture & Implementation Specifications

3.1 Interface Contracts (src/thermodynamics/state_validator.ts)

The interface contract enforces strict typing for discrepancy reports and validator signatures:

```
import { StateVector } from './state_vector';

export interface IStateDiscrepancyReport {
  readonly timestamp: number;
  readonly absoluteDiscrepancy: Map<string, number>;
  readonly relativeDiscrepancy: Map<string, number>;
  readonly totalMassDelta: number;
  readonly energyViolationDetected: boolean;
  readonly entropyDelta: number;
}

export interface IStateValidator {
  evaluateDiscrepancy(
    actual: StateVector,
    expected: StateVector
  ): IStateDiscrepancyReport;
}
```

3.2 Concrete Evaluator Implementation

The concrete implementation computes absolute and relative discrepancies across all state keys, aggregating mass inventories and testing energy constraints:

```
import { StateVector } from './state_vector';
import { IStateValidator, IStateDiscrepancyReport } from './state_validator';
```

```

export class StateDiscrepancyEvaluator implements IStateValidator {
  private tolerance: number;

  constructor(tolerance: number = 1e-6) {
    this.tolerance = tolerance;
  }

  public evaluateDiscrepancy(
    actual: StateVector,
    expected: StateVector
  ): IStateDiscrepancyReport {
    const absoluteDiscrepancy = new Map<string, number>();
    const relativeDiscrepancy = new Map<string, number>();

    let totalMassDelta = 0;
    let energyViolationDetected = false;
    let entropyDelta = 0;

    const actualMap = actual.toMap();
    const expectedMap = expected.toMap();
    const allKeys = new Set([...actualMap.keys(), ...expectedMap.keys()]);

    for (const key of allKeys) {
      const actVal = actualMap.get(key) ?? 0;
      const expVal = expectedMap.get(key) ?? 0;
      const delta = actVal - expVal;

      absoluteDiscrepancy.set(key, Math.abs(delta));
      const rel = expVal !== 0 ? Math.abs(delta / expVal) : Math.abs(delta);
      relativeDiscrepancy.set(key, rel);

      if (
        key.includes('mass') ||
        key.includes('carbon') ||
        key.includes('nitrogen') ||
        key.includes('phosphorus') ||
        key.includes('water')
      ) {
        totalMassDelta += delta;
      }

      if (key.includes('energy') && delta > this.tolerance) {
        energyViolationDetected = true;
      }
    }

    if (Math.abs(totalMassDelta) > this.tolerance) {
      energyViolationDetected = true;
    }

    entropyDelta = actual.getEntropy() - expected.getEntropy();

    return {
      timestamp: Date.now(),
      absoluteDiscrepancy,
      relativeDiscrepancy,
    };
  }
}

```

```
        totalMassDelta ,
        energyViolationDetected ,
        entropyDelta
    };
}
```

4 Verification and Test Plan

Unit tests established in `tests/sprint_082.test.ts` validate the evaluator against controlled perturbation vectors:

1. **Balanced State:** Asserts $\Delta M_{\text{total}} = 0$ and `energyViolationDetected = false`.
2. **Mass Imbalance:** Asserts that unmodeled mass additions exceeding 10^{-6} correctly trip `energyViolationDetected = true`.
3. **Energy Breach:** Asserts that spontaneous energy generation is isolated and flagged.

5 Conclusion

Sprint 082 successfully establishes the thermodynamic state vector discrepancy evaluator within the *Web of Life* platform. By embedding strict First and Second Law verification into runtime monad evaluations, the simulation framework guarantees physical fidelity across complex ecological interactions.

For updates, documentation, and source code access, please visit the repository: <https://github.com/pascalranoroarijaona/WebOfLife>.